The Open Source Audit

A Capstone Project for OSS NGMC Course

Chosen Software: Python

Student Name	Aryan Shandilya
Registration Number	24MIM10151
Date of Submission	28 March 2026
Course	Open Source Software

Table of Contents

Part A — Origin and Philosophy 3

A1. The Problem This Software Was Created to Solve 3

A2. The License — What It Actually Says 5

A3. The Ethics of Open Source 7

Part B — Linux Footprint 9

Part C — The FOSS Ecosystem 11

Part D — Open Source vs Proprietary 13

Shell Scripts Documentation 15

Part A — Origin and Philosophy

Open source software represents one of the most significant collaborative achievements in the history of computing. Among the myriad of open-source projects that have shaped our digital world, Python stands as a remarkable example of how a simple idea, nurtured by community collaboration, can transform into a global phenomenon that powers everything from introductory programming courses to artificial intelligence research. This section explores the origins of Python, examines its licensing framework, and reflects on the ethical dimensions of open-source software development.

A1. The Problem This Software Was Created to Solve

To understand why Python was created, we must transport ourselves to the computing landscape of the late 1980s. This was an era when programming was largely the domain of specialists, and the tools available to programmers were often frustratingly complex. The dominant languages of the time—C, C++, and various assembly languages—required programmers to manage memory manually, navigate cryptic syntax, and invest considerable time in understanding low-level system details before they could accomplish meaningful tasks.

Guido van Rossum, a Dutch programmer working at the Centrum Wiskunde & Informatica (CWI) in the Netherlands, experienced these frustrations firsthand. In 1989, during the Christmas holidays, van Rossum found himself with time on his hands. He had been involved with the ABC programming language project, a language designed to be easy to learn and use, but which had ultimately failed to gain widespread adoption. Van Rossum saw both the potential and the limitations of ABC—it had excellent ideas about simplicity and readability, but suffered from implementation problems and lack of extensibility.

The problem that Python was created to solve was multifaceted. First, there was the barrier to entry problem: existing programming languages were unnecessarily difficult for beginners and even experienced programmers often struggled with arcane syntax. Second, there was the productivity problem: programmers spent more time wrestling with language quirks than solving actual problems. Third, there was the extensibility problem: many languages that claimed to be simple lacked the ability to connect with existing systems and libraries written in other languages.

Van Rossum described his motivation in his foreword to the Python Cookbook: 'I was looking for a hobby project to keep me occupied during the holidays. My office would be closed, I had a home computer, and not much else on my hands. I decided to write an interpreter for the kind of scripting language I had been thinking about. I chose Python as a working title, being in a slightly irreverent mood and a big fan of Monty Python's Flying Circus.' This humble beginning, as a hobby project, would eventually reshape the programming landscape.

What made Python's origin story particularly significant was that van Rossum chose not to keep his creation proprietary. In an era when software companies were fiercely protecting their intellectual property, van Rossum released Python as open-source software from the very beginning. This decision was not merely practical—it was philosophical. Van Rossum believed that a programming language, as a fundamental tool for human expression and problem-solving, should belong to its community of users, not to a single entity that could restrict its use or modification.

The context of this decision matters enormously. In the late 1980s and early 1990s, the free software movement led by Richard Stallman was gaining momentum, but it was still largely confined to system-level tools like compilers and text editors. Van Rossum's decision to make Python open source was part of a broader shift in thinking about how programming tools should be developed and distributed. He understood that a programming language would only reach its full potential if users could examine, modify, and extend it freely.

A2. The License — What It Actually Says

The Python Software Foundation License (PSF License) represents a distinctive approach to open-source licensing that reflects both practical considerations and philosophical commitments. Understanding this license requires us to examine not just its legal text, but its implications for users, developers, and the broader software ecosystem.

The Four Freedoms of Free Software, as defined by the Free Software Foundation, provide a useful framework for evaluating any open-source license. These freedoms are: the freedom to run the program for any purpose (Freedom 0); the freedom to study how the program works and change it (Freedom 1); the freedom to redistribute copies (Freedom 2); and the freedom to distribute copies of your modified versions (Freedom 3). The PSF License grants all four of these freedoms, but with specific conditions that differ from other common licenses.

Under the PSF License, Python source code may be freely used, modified, and distributed, including in proprietary software, without requiring the distributor to make their modifications available to the public. This places Python in the category of permissive licenses, similar to the MIT License and BSD licenses, rather than copyleft licenses like the GNU General Public License (GPL). The critical distinction lies in what happens when someone modifies Python and incorporates it into their own software. With the GPL, any distributed software that incorporates GPL-licensed code must itself be released under the GPL, creating a viral effect that ensures software freedom propagates downstream. The PSF License, by contrast, allows modified versions to be kept proprietary if the distributor chooses.

The question of whether a company can take Python, modify it, and sell it without sharing their changes has a nuanced answer. Yes, technically, a company could incorporate Python into a proprietary product and sell that product without releasing their modifications to the Python interpreter itself. However, they must still preserve the copyright notices and license terms in any Python code they distribute, and they cannot claim ownership of the original Python code. This has led to Python being used extensively in commercial products, from video games to financial trading systems, without friction between commercial interests and community development.

The difference between GPL and MIT-style licensing (including the PSF License) represents a fundamental philosophical divide in the open-source community. The GPL takes a maximalist approach to software freedom: it ensures that any software that incorporates GPL code remains free forever, creating a protected commons of software that cannot be enclosed. Permissive licenses like MIT and PSF take a minimalist approach: they grant maximum freedom to downstream users, even if that freedom includes the freedom to make the software non-free. When asked which license I would choose for something I built, the answer depends on context. For a fundamental infrastructure tool like a programming language or operating system component, a permissive license makes sense because it encourages maximum adoption. For an application or library where I wanted to ensure ongoing community benefit, the GPL's protections

would be more appropriate.

The distinction between 'free as in free beer' and 'free as in freedom' is powerfully illustrated by Python's licensing history. Python is free in both senses: it costs nothing to download and use, and it grants users substantial freedoms over their computing. However, the PSF License makes Python 'free as in freedom' in a way that permits commercial use without requiring payment. Companies like Google, Facebook, and Netflix use Python extensively in their infrastructure, and they have contributed back to the Python community not because the license forces them to, but because it makes business sense to improve a tool they rely upon. This demonstrates that the 'free as in freedom' aspect can create sustainable open-source ecosystems even without copyleft enforcement.

A3. The Ethics of Open Source

The question of whether all software should be open source invites us to examine the relationship between knowledge, power, and social responsibility. There are compelling arguments on both sides, and the position one takes often depends on what values one prioritizes and what one believes about the nature of software itself.

The argument that all software should be open source rests on several ethical foundations. First, there is the argument from transparency: in an age where software mediates increasingly critical aspects of human life—from medical devices to voting systems to financial markets—the ability to examine how software works is essential for informed consent and democratic accountability. When software is closed-source, users are forced to trust not just the software's creators, but everyone in the chain of distribution and everyone who might have compromised the software along the way. Second, there is the argument from efficiency: closed-source software inevitably leads to duplication of effort, as different organizations solve the same problems in isolation. The open-source model allows humanity to build upon shared foundations, concentrating effort on innovation rather than reinvention. Third, there is the argument from justice: when software encodes rules that affect people's lives—credit scoring algorithms, hiring systems, criminal justice risk assessments—those affected have a moral right to understand and challenge how they are being evaluated.

However, there are also legitimate arguments against mandating open-source software. The most significant concerns innovation incentives: if all software must be open source, companies may have reduced motivation to invest in software development, particularly in areas requiring substantial upfront investment. While the open-source movement has demonstrated that many types of software can be sustainably developed through community contribution, there remain categories of software—particularly specialized applications with limited user bases—where the commercial model drives necessary investment. Additionally, there are legitimate concerns about security: while open-source advocates argue that 'many eyes make bugs shallow,' the reality is that critical vulnerabilities can persist in open-source code for years, and making source code available also makes it easier for malicious actors to identify vulnerabilities. The Log4j vulnerability of 2021 illustrated this tension: the affected library was open source, yet a critical vulnerability went undetected for years, affecting millions of systems worldwide.

The question of whether it is ethical for companies to build profitable products entirely on top of free community labor requires us to distinguish between exploitation and legitimate value creation. Companies like Red Hat (now part of IBM) have built billion-dollar businesses on Linux, and major technology companies derive enormous value from open-source infrastructure. Is this ethical? The answer, I believe,

lies in whether these companies contribute back to the communities from which they benefit. Red Hat, for instance, employs hundreds of developers who contribute to the Linux kernel and other open-source projects. Google has released TensorFlow, Kubernetes, and countless other tools as open source. When companies participate in the open-source ecosystem as contributors rather than mere consumers, the relationship becomes symbiotic rather than parasitic. The ethical concern arises when companies take from open source without giving back—a pattern that the community increasingly recognizes and responds to through both social pressure and, in some cases, license changes.

The phrase 'standing on the shoulders of giants,' attributed to Isaac Newton but with roots reaching back to medieval scholars, captures something essential about the nature of knowledge and innovation. In software development, this principle manifests as the understanding that every programmer builds upon layers of abstraction created by others—the compiler, the operating system, the libraries, the networking protocols—all the way down to the hardware. Open source makes this foundation visible and accessible. Rather than undermining original innovation, open source enables it by providing robust, tested foundations upon which new ideas can be constructed. A developer using Python to implement a novel machine learning algorithm does not diminish their creativity because Python itself is open source; rather, their creativity is enabled by not having to reinvent basic programming infrastructure. The open-source model ensures that the 'giants'—the accumulated wisdom of the programming community—remain accessible to all, rather than being enclosed within corporate walls.

Part B — Linux Footprint

Understanding how Python lives on a Linux system provides insight into the practical realities of open-source software deployment. This section documents the technical aspects of Python's presence on Linux, based on installation and examination on a Linux system.

Installation Method

Python is typically available through the standard package repositories of all major Linux distributions, reflecting its status as a fundamental piece of modern computing infrastructure. On Debian-based systems like Ubuntu, Python can be installed using the Advanced Package Tool (APT) with the command 'sudo apt install python3'. On Red Hat-based systems like Fedora or CentOS, the equivalent command using DNF or YUM is 'sudo dnf install python3'. Alternatively, Python can be compiled from source code downloaded from the official Python website, which provides maximum control over build options but requires more technical expertise and time.

The prevalence of Python in Linux distributions is such that most modern Linux systems come with Python pre-installed. This reflects Python's role as a dependency for many system utilities and applications—the package managers themselves, system configuration tools, and desktop environments often rely on Python. This ubiquity is a testament to the success of Python's design philosophy: a language so useful and well-designed that it becomes infrastructure.

Key Directories

Python's installation on Linux follows the Filesystem Hierarchy Standard (FHS), which defines the directory structure and directory contents in Unix-like operating systems. The main Python interpreter binary is typically located at /usr/bin/python3, which is a symbolic link to the version-specific binary such as /usr/bin/python3.11 or /usr/bin/python3.12. The standard library is installed under /usr/lib/python3.x/, where x represents the minor version number. Configuration files are found in /etc/python3.x/, including the pythonrc file and site-specific configuration.

Directory	Purpose	Example
/usr/bin/	Binary executables	/usr/bin/python3
/usr/lib/python3.x/	Standard library modules	/usr/lib/python3.11/
/usr/local/lib/python3.x/	Locally installed packages	User-installed libraries
/etc/python3.x/	Configuration files	pythonrc, site.cfg
~/.local/lib/	User-installed packages (pip)	Per-user Python packages
/var/log/	Log files (if applicable)	Application logs

Table 1: Python Directory Structure on Linux

User and Group

On most Linux systems, Python does not run as a dedicated service user; instead, it executes under the credentials of the user who invokes it. This design reflects Python's nature as a general-purpose tool rather than a system service. However, when Python applications are deployed as system services—for example, a Django web application or a data processing pipeline—they typically run under a dedicated user account created specifically for that application. This follows the principle of least privilege: each service should have only the permissions necessary for its operation, limiting the potential damage if the service is compromised.

The security implications of this arrangement are significant. Because Python scripts execute with the permissions of the invoking user, a malicious Python script can access any file or resource that the user can access. This is why it's considered a best practice to run services under dedicated, unprivileged accounts rather than as root. The pip package manager, which installs Python packages from the Python Package Index (PyPI), should typically not be run as root unless installing packages system-wide is explicitly required, as malicious packages could potentially compromise system security.

Service Management

While the Python interpreter itself is not typically managed as a system service, Python applications can be configured to run as services using systemd, the standard service manager on most modern Linux distributions. A typical systemd service unit file for a Python application specifies the path to the Python interpreter, the script to execute, the user under which to run, and dependencies on other services like network availability.

The standard systemd commands apply: 'sudo systemctl start myapp' to start the service, 'sudo systemctl stop myapp' to stop it, 'sudo systemctl restart myapp' to restart it, and 'sudo systemctl status myapp' to check its status. The command 'sudo systemctl enable myapp' configures the service to start automatically at boot, while 'sudo systemctl disable myapp' prevents automatic startup. This integration with systemd demonstrates how Python applications can be first-class citizens in the Linux service ecosystem.

Update Model

Security patches and updates for Python reach Linux systems through several channels, reflecting the distributed nature of open-source software distribution. For the majority of users, Python updates come through the distribution's package manager—apt, dnf, yum, or pacman—maintained by the distribution's packaging team. These teams monitor security advisories from the Python Software Foundation and issue updated packages when vulnerabilities are discovered. The advantage of this model is that users receive tested, integrated updates; the disadvantage is that there may be a delay between upstream Python releases and distribution package updates.

For users who require the absolute latest Python versions or who have specific version requirements, alternative installation methods exist. The deadsnakes PPA (Personal Package Archive) provides newer Python versions for Ubuntu users. The pyenv tool allows users to install and manage multiple Python versions in their home directory without requiring system administrator privileges. Containerization technologies like Docker enable running specific Python versions in isolated environments. This diversity

of update pathways ensures that users with different needs can all be accommodated, a hallmark of the open-source ecosystem's flexibility.

Part C — The FOSS Ecosystem

No open-source project exists in isolation. Python's development and success have been shaped by countless other projects, and Python itself has enabled the creation of countless more. Understanding this ecosystem reveals how open-source software development creates value that extends far beyond any single project.

Dependencies and Foundations

Python's implementation depends on several foundational open-source technologies. The C programming language and the GNU Compiler Collection (GCC) or LLVM/Clang provide the compilation infrastructure that transforms Python's C source code into executable binaries. The Python interpreter itself is written primarily in C, making it a bridge between low-level system programming and high-level application development. Build tools like GNU Make and Autoconf manage the complex process of compiling Python across different platforms and architectures.

At runtime, Python interacts with numerous system libraries: the C standard library for basic operations, OpenSSL for cryptographic functions, zlib for compression, and various others depending on the platform and configuration. These dependencies illustrate a crucial aspect of open-source development: even projects that seem self-contained actually rest upon layers of community-maintained infrastructure. When security vulnerabilities are discovered in OpenSSL or zlib, for instance, Python applications may be affected even though the vulnerability exists in a dependency rather than Python itself.

What Python Has Enabled

Perhaps no aspect of Python's impact is more dramatic than its role in the data science and machine learning revolution. Libraries like NumPy, Pandas, and Matplotlib transformed Python from a general-purpose scripting language into the dominant platform for scientific computing. Building on this foundation, frameworks like TensorFlow, PyTorch, and scikit-learn made Python the language of choice for machine learning and artificial intelligence. These tools, all open-source, have accelerated research and democratized access to techniques that were previously confined to well-funded laboratories.

The web development ecosystem built on Python is equally impressive. Django, a full-featured web framework, powers sites handling millions of requests per day, including Instagram and Pinterest. Flask provides a lightweight alternative for smaller applications and APIs. FastAPI, a more recent entrant, leverages Python's type hints to provide automatic API documentation and validation. These frameworks handle routing, database connectivity, authentication, and countless other concerns, enabling developers to focus on application logic rather than reinventing infrastructure.

Domain	Key Projects	Impact
Data Science	NumPy, Pandas, Matplotlib	Standard for scientific computing
Machine Learning	TensorFlow, PyTorch, scikit-learn	Dominant ML platform

Web Development	Django, Flask, FastAPI	Millions of websites
DevOps	Ansible, SaltStack, Fabric	Infrastructure automation
Testing	pytest, unittest, nose	Quality assurance

Table 2: Python Ecosystem by Domain

Relation to LAMP Stack

The LAMP stack—Linux, Apache, MySQL, PHP—defined web development for a generation. Python's relationship to this stack is interesting: while Python can replace PHP as the P in LAMP (some jokingly call it LAMP with a Python P), Python's ecosystem has evolved in directions that sometimes complement and sometimes compete with traditional LAMP. Django, for instance, typically includes its own database abstraction layer and can work with PostgreSQL as readily as MySQL. Modern Python web applications often use application servers like Gunicorn or uWSGI behind Nginx, rather than Apache's mod_python, reflecting architectural shifts toward containerization and microservices. Python's versatility means it can slot into traditional architectures or support entirely new approaches.

Community and Governance

The Python community is renowned in the software world for its emphasis on readability, simplicity, and inclusion. This culture is codified in PEP 20, 'The Zen of Python,' which includes aphorisms like 'Readability counts,' 'Simple is better than complex,' and 'There should be one—and preferably only one—obvious way to do it.' These principles shape not only the language's design but also the culture of its community.

Governance of Python has evolved over its history. For many years, Guido van Rossum served as the 'Benevolent Dictator for Life' (BDFL), making final decisions about language direction. In 2018, van Rossum stepped down from this role, and Python transitioned to a Steering Council model. Five elected council members oversee the language's development, with major changes proposed through Python Enhancement Proposals (PEPs) and discussed openly on mailing lists and GitHub. This governance structure balances efficiency with community input, ensuring that Python evolves in ways that serve its diverse user base.

The community maintains numerous communication channels: the python-dev mailing list for core development discussions, python-ideas for broader language proposals, the Python Software Foundation's forums for governance matters, and numerous special-interest groups (SIGs) for specific domains like education, web development, and scientific computing. Annual conferences like PyCon provide opportunities for face-to-face collaboration and knowledge sharing. The Python Package Index (PyPI), hosting over 400,000 packages, represents one of the largest software repositories in existence, maintained by a combination of paid PSF staff and volunteers.

Part D — Open Source vs Proprietary

A critical analysis of Python must honestly assess its strengths and weaknesses relative to proprietary alternatives. This comparison examines multiple dimensions without assuming that open source is inherently superior in every case.

Dimension	Python (Open Source)	Proprietary Alternative (MATLAB)
Cost	Free to download, use, and distribute. No licensing fees for development or deployment.	Commercial licenses cost hundreds to thousands of dollars annually. Toolboxes require additional purchase.
Security Audit	Anyone can examine source code. Vulnerabilities identified by global community. CVEs publicly disclosed.	Source code not available for inspection. Security through obscurity. Users must trust vendor.
Support	Community forums, Stack Overflow, documentation. Commercial support available from vendors.	Professional support from vendor. Guaranteed response times for paying customers.
Freedom to Modify	Full freedom to modify, extend, and distribute changes. Can fix bugs yourself.	Cannot modify source code. Must wait for vendor to fix issues.
Community vs Corporate	Community-driven development. Decisions made transparently through PEP process.	Corporate-controlled roadmap. Features prioritized by business considerations.

Table 3: Open Source vs Proprietary Comparison (Python vs MATLAB)

Analysis and Verdict

The comparison between Python and its proprietary alternatives reveals a nuanced picture where the optimal choice depends heavily on context. For scientific computing and data analysis—a domain where MATLAB has traditionally been strong—Python has emerged as a compelling alternative that matches or exceeds MATLAB's capabilities while eliminating licensing costs and vendor dependency. The combination of NumPy, SciPy, Matplotlib, and related libraries provides functionality comparable to MATLAB's core, while the broader Python ecosystem offers capabilities that MATLAB cannot match, from web development to machine learning to automation scripting.

However, there remain scenarios where proprietary alternatives have advantages. MATLAB's integrated development environment provides a polished, cohesive experience that can be easier for beginners to navigate than Python's more fragmented tooling. MATLAB's toolboxes, while expensive, are professionally maintained and documented, with guaranteed compatibility and support. In corporate

environments where risk aversion is high and budget constraints are low, the accountability provided by a commercial vendor can be valuable. Organizations with existing MATLAB codebases and expertise face significant switching costs that may not be justified by Python's advantages.

For real-world deployment, my recommendation would depend on the specific context. For new projects, especially those with budget constraints or requirements for long-term maintainability, Python is the superior choice. Its open-source nature ensures that code will remain accessible and modifiable regardless of any single company's business decisions. The vast ecosystem of libraries and the active community provide resources that no proprietary vendor can match. For organizations with existing investments in proprietary tools, a gradual transition strategy—using Python for new components while maintaining legacy systems—often makes sense. Python's ability to interoperate with other languages and systems facilitates this hybrid approach.

Would I contribute to Python? Absolutely. Contributing to Python—whether through code, documentation, bug reports, or community support—represents an opportunity to give back to a project that has given so much to the world. The open-source model succeeds because participants recognize that their contributions benefit not only others but also themselves, as improvements to the platform raise the baseline for everyone. Contributing to Python would be both a practical way to improve a tool I use and a meaningful participation in the broader open-source movement.

Shell Scripts Documentation

As part of this capstone project, five shell scripts have been developed to demonstrate practical Linux skills and understanding of shell scripting concepts. Each script is documented below with its purpose, concepts used, and execution instructions.

Script 1: System Identity Report

This script generates a comprehensive welcome screen displaying system information including Linux distribution name, kernel version, current user, home directory, system uptime, and open-source license information. The script demonstrates the use of variables, command substitution with $() syntax, echo for output formatting, and basic conditional statements for distribution detection. It checks Python installation status as part of the audit project, providing a complete picture of the system on which the Python audit is being conducted.

```bash
#!/bin/bash
# ==============================================================================
# Script 1: System Identity Report
# Author: Aryan Shandilya | Course: Open Source Software
# Registration No: 24MIM10151
# Description: This script introduces the Linux system like a welcome screen,
#              displaying system information including distribution, kernel,
#              user details, uptime, and open-source license information.
# Concepts Used: variables, echo, command substitution ($()), output formatting
# ==============================================================================

# --- Variables ---
STUDENT_NAME="Aryan Shandilya"          # Student name for display
SOFTWARE_CHOICE="Python"              # Chosen open-source software for audit
REGISTRATION_NO="24MIM10151"          # Student registration number

# --- System Information Collection ---
# Using command substitution $() to capture command output into variables
KERNEL=$(uname -r)                    # Kernel version using uname command
USER_NAME=$(whoami)                   # Current logged-in username
UPTIME=$(uptime -p)                   # System uptime in human-readable format
CURRENT_DATE=$(date '+%A, %d %B %Y')  # Current date with day name
CURRENT_TIME=$(date '+%H:%M:%S %Z')   # Current time with timezone
HOME_DIR=$(echo $HOME)                # Home directory of current user
HOSTNAME=$(hostname)                  # System hostname

# --- Distribution Detection ---
# Check for different distribution files to identify the Linux distro
if [ -f /etc/os-release ]; then
    # Most modern distributions have /etc/os-release
    DISTRO=$(grep "^PRETTY_NAME=" /etc/os-release | cut -d'"' -f2)
elif [ -f /etc/lsb-release ]; then
    # Ubuntu and derivatives
    DISTRO=$(grep "DISTRIB_DESCRIPTION=" /etc/lsb-release | cut -d'"' -f2)
elif [ -f /etc/redhat-release ]; then
```

```
    # Red Hat and derivatives
    DISTRO=$(cat /etc/redhat-release)
else
    DISTRO="Unknown Linux Distribution"
fi

# --- License Information ---
# The OS license message - Linux kernel and most components are GPL licensed
LICENSE_MSG="This operating system is primarily covered under GPL (GNU General Public License)
The Linux Kernel is licensed under GPL v2, ensuring freedom to use, study, modify, and distrib

# --- Display Section ---
# Clear screen for a clean welcome display
clear

# Print decorative header
echo "╔═══════════════════════════════════════════════════════════════════╗"
echo "│                    OPEN SOURCE AUDIT - SYSTEM IDENTITY            │"
echo "╠═══════════════════════════════════════════════════════════════════╣"

# Display student information
echo "│  Student Name    : $STUDENT_NAME"
echo "│  Registration No : $REGISTRATION_NO"
echo "│  Software Audit  : $SOFTWARE_CHOICE"
echo "╠═══════════════════════════════════════════════════════════════════╣"

# Display system information
echo "│                         SYSTEM INFORMATION                        │"
echo "╠═══════════════════════════════════════════════════════════════════╣"
echo "│  Hostname        : $HOSTNAME"
echo "│  Distribution    : $DISTRO"
echo "│  Kernel Version  : $KERNEL"
echo "│  Current User    : $USER_NAME"
echo "│  Home Directory  : $HOME_DIR"
echo "│  System Uptime   : $UPTIME"
```

```
echo "│  Current Date    : $CURRENT_DATE"
echo "│  Current Time    : $CURRENT_TIME"
echo "├──────────────────────────────────────────────────────────────────────┤"

# Display license information
echo "│                        LICENSE INFORMATION                           │"
echo "├──────────────────────────────────────────────────────────────────────┤"
echo "│  $LICENSE_MSG"
echo "└──────────────────────────────────────────────────────────────────────┘"

# --- Python Installation Check ---
# Since Python is our chosen software, check its installation
echo ""
echo "══════════════════════════════════════════════════════════════════════"
echo "                     PYTHON INSTALLATION STATUS"
echo "══════════════════════════════════════════════════════════════════════"

# Check for Python 3
if command -v python3 &> /dev/null; then
    PYTHON3_VERSION=$(python3 --version 2>&1)
    PYTHON3_PATH=$(which python3)
    echo "✓ $PYTHON3_VERSION is installed"
    echo "  Path: $PYTHON3_PATH"
else
    echo "✗ Python 3 is NOT installed"
fi

# Check for Python 2 (legacy)
if command -v python2 &> /dev/null; then
    PYTHON2_VERSION=$(python2 --version 2>&1)
    PYTHON2_PATH=$(which python2)
    echo "✓ $PYTHON2_VERSION is installed"
    echo "  Path: $PYTHON2_PATH"
else
    echo "✗ Python 2 is NOT installed (legacy version)"
```

#

```
fi

echo ""
echo "═══════════════════════════════════════════════════════════════"
echo "Script execution completed at $(date '+%H:%M:%S on %d/%m/%Y')"
echo "═══════════════════════════════════════════════════════════════"

# Exit with success status
exit 0
```

GIVEN SCREENSHOT:

```
╔═══════════════════════════════════════════════════════════════╗
                OPEN SOURCE AUDIT - SYSTEM IDENTITY
╚═══════════════════════════════════════════════════════════════╝
  Student Name     : Aryan Shandilya
  Registration No  : 24MIM10151
  Software Audit   : Python
╔═══════════════════════════════════════════════════════════════╗
                     SYSTEM INFORMATION
╚═══════════════════════════════════════════════════════════════╝
  Hostname         : Lenovo
  Distribution     : Ubuntu 24.04.3 LTS
  Kernel Version   : 6.6.87.2-microsoft-standard-WSL2
  Current User     : aryan
  Home Directory   : /home/aryan
  System Uptime    : up 0 minutes
  Current Date     : Saturday, 28 March 2026
  Current Time     : 14:10:07 UTC
╔═══════════════════════════════════════════════════════════════╗
                     LICENSE INFORMATION
╚═══════════════════════════════════════════════════════════════╝
  This operating system is primarily covered under GPL (GNU General Public License).
The Linux Kernel is licensed under GPL v2, ensuring freedom to use, study, modify, and distribute.

═══════════════════════════════════════════════════════════════
                PYTHON INSTALLATION STATUS
═══════════════════════════════════════════════════════════════
✓ Python 3.12.3 is installed
  Path: /usr/bin/python3
✗ Python 2 is NOT installed (legacy version)

═══════════════════════════════════════════════════════════════
Script execution completed at 14:10:07 on 28/03/2026
═══════════════════════════════════════════════════════════════
```

Script 2: FOSS Package Inspector

This script checks whether Python is installed on the system, retrieves version and license information, and uses a case statement to display philosophical notes about various FOSS packages. It demonstrates if-then-else conditional logic, case statements for multiple-choice handling, package manager detection (dpkg/rpm), and pipe operations with grep for text filtering. The script provides both practical.

```
#!/bin/bash
# ==============================================================================
# Script 2: FOSS Package Inspector
# Author: Aryan Shandilya | Course: Open Source Software      You, 14 minutes ago • Fi
# Registration No: 24MIM10151
# Description: This script checks whether the chosen software (Python) is
#              installed on the system, finds its version, and uses a case
#              statement to print philosophical notes about various FOSS packages.
# Concepts Used: if-then-else, case statement, command substitution,
#                rpm/dpkg commands, pipe with grep, command -v
# ==============================================================================

# --- Configuration Variables ---
PACKAGE="python3"                     # Package to inspect (Python 3)
AUTHOR="Aryan Shandilya"
REG_NO="24MIM10151"

# --- Display Header ---
echo "╔═══════════════════════════════════════════════════════════════════════╗"
echo "│                    FOSS PACKAGE INSPECTOR                             │"
echo "│                    Author: $AUTHOR | Reg No: $REG_NO"
echo "╚═══════════════════════════════════════════════════════════════════════╝"
echo ""

# --- Function to check package using apt/dpkg (Debian/Ubuntu) ---
check_dpkg() {
    local pkg=$1
    # dpkg -l lists installed packages, grep filters for exact match
    if dpkg -l | grep -q "^[[:space:]]*ii[[:space:]]*${pkg}"; then
        return 0  # Package found
    else
        return 1  # Package not found
    fi
}
```

```
35
36  # --- Function to check package using rpm (Red Hat/Fedora/CentOS) ---
37  check_rpm() {
38      local pkg=$1
39      # rpm -q queries if package is installed
40      if rpm -q "$pkg" &>/dev/null; then
41          return 0  # Package found
42      else
43          return 1  # Package not found
44      fi
45  }
46
47  # --- Function to get package info from dpkg ---
48  get_dpkg_info() {
49      local pkg=$1
50      echo "--- Package Information (dpkg) ---"
51      # Extract version, license, and description using dpkg and grep
52      dpkg -l "$pkg" 2>/dev/null | tail -1 | awk '{print "Package: " $2 "\nVersion: " $3}'
53      # Get description from apt-cache
54      apt-cache show "$pkg" 2>/dev/null | grep -E "^Description:" | head -1
55  }
56
57  # --- Function to get package info from rpm ---
58  get_rpm_info() {
59      local pkg=$1
60      echo "--- Package Information (rpm) ---"
61      # rpm -qi provides detailed package information
62      rpm -qi "$pkg" 2>/dev/null | grep -E "Name|Version|License|Summary"
63  }
64
65  # --- Determine Package Manager and Check Installation ---
66  echo "═══════════════════════════════════════════════════════════"
67  echo "Checking installation status for: $PACKAGE"
68  echo "═══════════════════════════════════════════════════════════"
69
```

```
# Detect which package manager is available
if command -v dpkg &>/dev/null; then
    PKG_MANAGER="dpkg"
    if check_dpkg "$PACKAGE"; then
        INSTALLED=true
    else
        INSTALLED=false
    fi
elif command -v rpm &>/dev/null; then
    PKG_MANAGER="rpm"
    if check_rpm "$PACKAGE"; then
        INSTALLED=true
    else
        INSTALLED=false
    fi
else
    # Fallback: check using command -v (checks if command exists in PATH)
    PKG_MANAGER="command"
    if command -v "$PACKAGE" &>/dev/null; then
        INSTALLED=true
    else
        INSTALLED=false
    fi
fi

# --- Display Installation Status and Information ---
if [ "$INSTALLED" = true ]; then
    echo "✓ $PACKAGE is INSTALLED on this system"
    echo ""

    # Get detailed information based on package manager
    case $PKG_MANAGER in
        "dpkg")
            get_dpkg_info "$PACKAGE"
            ;;
```

```
        "rpm")
            get_rpm_info "$PACKAGE"
            ;;
        "command")
            # Use generic commands to get version
            echo "--- Package Information ---"
            echo "Command path: $(which $PACKAGE)"
            $PACKAGE --version 2>&1 | head -3
            ;;
    esac
else
    echo "✗ $PACKAGE is NOT INSTALLED on this system"
    echo ""
    echo "To install $PACKAGE:"
    if [ "$PKG_MANAGER" = "dpkg" ]; then
        echo "  sudo apt update && sudo apt install $PACKAGE"
    elif [ "$PKG_MANAGER" = "rpm" ]; then
        echo "  sudo dnf install $PACKAGE   (or: sudo yum install $PACKAGE)"
    else
        echo "  Please use your distribution's package manager"
    fi
fi

# --- Case Statement: Philosophical Notes about FOSS Packages ---
echo ""
echo "═══════════════════════════════════════════════════════════════"
echo "                    FOSS PHILOSOPHY CORNER"
echo "═══════════════════════════════════════════════════════════════"
```

```
echo ""
echo "Philosophical notes about open-source software (selected packages):"
echo ""

# Case statement to print philosophy notes based on package name
case "$PACKAGE" in
    python | python3)
        echo "🐍 PYTHON: 'Readability counts. Simple is better than complex.'"
        echo "   Python embodies the open-source spirit - a language built by community,"
        echo "   for community. Created by Guido van Rossum as a hobby project, it grew"
        echo "   into one of the world's most influential programming languages."
        echo "   The PSF License ensures it remains free and open for everyone."
        ;;
    httpd | apache2)
        echo "🌐 APACHE: The web server that built the open internet."
        echo "   Apache HTTP Server has powered the web since 1995, proving that"
        echo "   open-source can dominate even the most critical infrastructure."
        ;;
    mysql | mariadb)
        echo "🗄  MySQL: Open source at the heart of millions of applications."
        echo "   From LAMP stacks to enterprise systems, MySQL demonstrates how"
        echo "   open-source databases can compete with proprietary alternatives."
        ;;
    firefox)
        echo "🦊 FIREFOX: A nonprofit fighting for an open web."
        echo "   Mozilla Firefox proves you don't need corporate profit motives"
        echo "   to build world-class software that respects user privacy."
        ;;
    vlc)
        echo "🎬 VLC: Plays anything - built by students in Paris."
        echo "   VideoLAN started as a student project and became the universal"
        echo "   media player, showing how academic freedom enables innovation."
        ;;
    git)
        echo "📦 GIT: The tool Linus built when proprietary failed him."
        echo "   Created in 2005 when BitKeeper revoked free access. Git shows"
```

```
        echo "   how necessity and open-source philosophy create powerful tools."
        ;;
    libreoffice)
        echo "📄 LIBREOFFICE: Born from a community fork - a real lesson."
        echo "   LibreOffice emerged when OpenOffice users forked the project,"
        echo "   demonstrating the GPL's power to protect community interests."
        ;;
    linux | kernel)
        echo "🐧 LINUX KERNEL: The foundation everything else runs on."
        echo "   Linus Torvalds' 'just for fun' project became the backbone of"
        echo "   modern computing - from phones to supercomputers."
        ;;
    *)
        echo "❓ $PACKAGE: Every open-source project has a story worth telling."
        echo "   Open-source software represents the collective wisdom of thousands"
        echo "   of contributors who chose to share rather than hoard knowledge."
        ;;
esac

# --- Extended Case Statement for Additional Packages ---
echo ""
echo "Additional FOSS philosophy notes:"
echo ""

# Loop through additional packages to show more philosophy notes
for pkg in git firefox vlc; do
    case $pkg in
        git)
            echo "  • Git: Distributed version control - because code freedom matters"
            ;;
        firefox)
```

```
            echo "  • Firefox: Privacy-focused browsing - the web should remain open"
            ;;
        vlc)
            echo "  • VLC: Universal media playback - codecs should be free"
            ;;
    esac
done

echo ""
echo "═══════════════════════════════════════════════════════════"
echo "Script execution completed at $(date '+%H:%M:%S')"
echo "═══════════════════════════════════════════════════════════"

exit 0
```

GIVEN SCREENSHOT:

```
Script execution completed at 14:18:43 on 28/03/2026
═══════════════════════════════════════════════════════════════════════

C:\Users\aryan\OneDrive\Desktop\CODING\CLONED REPO's\OSS-project-24MIM10151>wsl bash ./script2_foss_package_inspector.sh
╔═══════════════════════════════════════════════════════════════════════╗
                    FOSS PACKAGE INSPECTOR
                    Author: Aryan Shandilya │ Reg No: 24MIM10151
╚═══════════════════════════════════════════════════════════════════════╝

═══════════════════════════════════════════════════════════════════════
Checking installation status for: python3
═══════════════════════════════════════════════════════════════════════
√ python3 is INSTALLED on this system

--- Package Information (dpkg) ---
Package: python3
Version: 3.12.3-0ubuntu2

═══════════════════════════════════════════════════════════════════════
                   FOSS PHILOSOPHY CORNER
═══════════════════════════════════════════════════════════════════════

Philosophical notes about open-source software (selected packages):

🐍 PYTHON: 'Readability counts. Simple is better than complex.'
   Python embodies the open-source spirit - a language built by community,
   for community. Created by Guido van Rossum as a hobby project, it grew
   into one of the world's most influential programming languages.
   The PSF License ensures it remains free and open for everyone.

Additional FOSS philosophy notes:

  • Git: Distributed version control - because code freedom matters
  • Firefox: Privacy-focused browsing - the web should remain open
  • VLC: Universal media playback - codecs should be free

═══════════════════════════════════════════════════════════════════════
Script execution completed at 14:23:40
═══════════════════════════════════════════════════════════════════════
```

Script 3: Disk and Permission Auditor

This script loops through important system directories and reports disk usage, owner, group, and permissions for each. It specifically checks Python-related directories as part of the audit and includes security analysis for world-writable directories. The script demonstrates for loops with arrays, du and df commands for disk information, ls -ld for directory permissions, awk and cut for text processing, and conditional tests using the test operator.

```bash
#!/bin/bash
# ==============================================================================
# Script 3: Disk and Permission Auditor
# Author: Aryan Shandilya | Course: Open Source Software
# Registration No: 24MIM10151
# Description: This script loops through important system directories and
#              reports disk usage, owner, and permissions. It also specifically
#              checks Python-related directories for the audit.
# Concepts Used: for loop, arrays, df command, ls -ld, awk, cut,
#                conditional tests (-d, -f), du command
# ==============================================================================

# --- Configuration ---
AUTHOR="Aryan Shandilya"      You, 21 minutes ago • First Update …
REG_NO="24MIM10151"
SOFTWARE="Python"

# --- Header Display ---
echo "╔═══════════════════════════════════════════════════════════════════════╗"
echo "│                    DISK AND PERMISSION AUDITOR                        │"
echo "│                    Author: $AUTHOR | Reg No: $REG_NO"
echo "╚═══════════════════════════════════════════════════════════════════════╝"
echo ""

# --- Define Directories to Audit ---
# Array of important system directories to check
SYSTEM_DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/usr/local" "/opt")

# --- Function to convert permission octal to symbolic notation explanation ---
explain_permissions() {
    local perms=$1
    local explanation=""

    # User permissions
    user_perm="${perms:1:3}"
    # Group permissions
```

```
    group_perm="${perms:4:3}"
    # Others permissions
    other_perm="${perms:7:3}"

    echo "User: $user_perm | Group: $group_perm | Others: $other_perm"
}

# --- Function to get directory information ---
get_dir_info() {
    local dir=$1

    # Check if directory exists using -d test
    if [ ! -d "$dir" ]; then
        echo "❌ $dir does not exist on this system"
        return 1
    fi

    # Get detailed listing using ls -ld (directory itself, not contents)
    # Using awk to extract specific fields
    local perms=$(ls -ld "$dir" 2>/dev/null | awk '{print $1}')
    local owner=$(ls -ld "$dir" 2>/dev/null | awk '{print $3}')
    local group=$(ls -ld "$dir" 2>/dev/null | awk '{print $4}')

    # Get directory size using du command
    # -s for summary, -h for human-readable format
    # 2>/dev/null suppresses permission denied errors
    local size=$(du -sh "$dir" 2>/dev/null | cut -f1)

    # Handle case where size couldn't be determined
    if [ -z "$size" ]; then
        size="Access Denied"
```

#!/

```
    fi

    # Display the information
    printf "%-15s | Size: %-10s | Owner: %-8s | Group: %-8s | Perms: %s\n" \
           "$dir" "$size" "$owner" "$group" "$perms"
}

# --- Section 1: System Directory Audit ---
echo "═══════════════════════════════════════════════════════════════════"
echo "                    SECTION 1: SYSTEM DIRECTORY AUDIT"
echo "═══════════════════════════════════════════════════════════════════"
echo ""
echo "Directory       | Size        | Owner     | Group     | Permissions"
echo "───────────────────────────────────────────────────────────────────"

# For loop through system directories
for DIR in "${SYSTEM_DIRS[@]}"; do
    get_dir_info "$DIR"
done

echo ""

# --- Section 2: Python-Specific Directories Audit ---
echo "═══════════════════════════════════════════════════════════════════"
echo "                    SECTION 2: PYTHON DIRECTORIES AUDIT"
echo "═══════════════════════════════════════════════════════════════════"
echo ""
```

```
# Python-specific directories to check
PYTHON_DIRS=(
    "/usr/lib/python3"
    "/usr/lib/python3.11"
    "/usr/lib/python3.12"
    "/usr/local/lib/python3"
    "/etc/python3"
    "$HOME/.local/lib/python"
    "$HOME/.local/bin"
)

echo "Python Installation Directories:"
echo "───────────────────────────────────────────────────────────────────"

# For loop through Python directories
for DIR in "${PYTHON_DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        # Directory exists - show details
        PERMS=$(ls -ld "$DIR" 2>/dev/null | awk '{print $1}')
        OWNER=$(ls -ld "$DIR" 2>/dev/null | awk '{print $3}')
        GROUP=$(ls -ld "$DIR" 2>/dev/null | awk '{print $4}')
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)

        printf "✓ %-30s | %s | %s:%s | %s\n" "$DIR" "$SIZE" "$OWNER" "$GROUP" "$PERMS"
    else
        printf "✗ %-30s | Not found\n" "$DIR"
    fi
done

echo ""
```

```
# --- Section 3: Python Binary and Executable Check ---
echo "═══════════════════════════════════════════════════════════════════"
echo "                    SECTION 3: PYTHON EXECUTABLES"
echo "═══════════════════════════════════════════════════════════════════"
echo ""

# Check Python executables
PYTHON_EXECUTABLES=("python" "python3" "python2" "pip" "pip3")

echo "Executable     | Location                          | Permissions"
echo "──────────────────────────────────────────────────────────────────"

for EXEC in "${PYTHON_EXECUTABLES[@]}"; do
    # Check if command exists
    if command -v "$EXEC" &>/dev/null; then
        LOCATION=$(which "$EXEC")
        # Check if it's a file (not alias or function)
        if [ -f "$LOCATION" ]; then
            PERMS=$(ls -l "$LOCATION" 2>/dev/null | awk '{print $1}')
            printf "%-15s | %-38s | %s\n" "$EXEC" "$LOCATION" "$PERMS"
        else
            printf "%-15s | %-38s | (alias/function)\n" "$EXEC" "$LOCATION"
        fi
    else
        printf "%-15s | Not installed\n" "$EXEC"
    fi
done

echo ""
```

```
# --- Section 4: Disk Space Summary ---
echo "═══════════════════════════════════════════════════════════════════════"
echo "                    SECTION 4: DISK SPACE SUMMARY"
echo "═══════════════════════════════════════════════════════════════════════"
echo ""

# Display filesystem disk usage
echo "Filesystem Disk Usage:"
echo ""
df -h | head -1  # Print header
df -h | grep -E "^/dev|^tmpfs|^overlay"  # Filter for relevant filesystems

echo ""

# --- Section 5: Security Analysis ---
echo "═══════════════════════════════════════════════════════════════════════"
echo "                    SECTION 5: SECURITY ANALYSIS"
echo "═══════════════════════════════════════════════════════════════════════"
echo ""

# Check for world-writable directories (potential security risk)
echo "Checking for world-writable directories (potential security concern):"
echo ""

WORLD_WRITABLE=0
for DIR in "${SYSTEM_DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        # Check if directory is world-writable
        if [ -w "$DIR" ] && [ "$(ls -ld "$DIR" | awk '{print $1}' | grep -c 'w.$')" -gt 0 ];
            # More precise check for world-writable
            PERMS=$(ls -ld "$DIR" | awk '{print $1}')
            if [[ "$PERMS" == *"w"* ]] && [[ "${PERMS: -1}" == "w" || "${PERMS: -1}" == "t" ]
                echo "⚠  $DIR is potentially world-writable: $PERMS"
                ((WORLD_WRITABLE++))
            fi
        fi
    fi
done

# Check /tmp specifically (expected to be world-writable with sticky bit)
if [ -d "/tmp" ]; then
    TMP_PERMS=$(ls -ld /tmp | awk '{print $1}')
    echo ""
    echo "Note: /tmp permissions: $TMP_PERMS"
    echo "  (World-writable with sticky bit is expected and secure for /tmp)"
fi

echo ""
if [ $WORLD_WRITABLE -eq 0 ]; then
    echo "✓ No unexpected world-writable directories found"
fi

echo ""
echo "═══════════════════════════════════════════════════════════════════════"
echo "Audit completed at $(date '+%H:%M:%S on %d/%m/%Y')"
echo "═══════════════════════════════════════════════════════════════════════"

exit 0
```

Given Screenshot:

```
╔═══════════════════════════════════════════════════════════════════╗
║                   DISK AND PERMISSION AUDITOR                     ╒
                    Author: Aryan Shandilya | Reg No: 24MIM10151
╚═══════════════════════════════════════════════════════════════════╝

═══════════════════════════════════════════════════════════════════
                  SECTION 1: SYSTEM DIRECTORY AUDIT
═══════════════════════════════════════════════════════════════════

Directory        | Size        | Owner      | Group     | Permissions
───────────────────────────────────────────────────────────────────
/etc             | Size: 4.4M       | Owner: root      | Group: root      | Perms: drwxr-xr-x
/var/log         | Size: 343M       | Owner: root      | Group: syslog    | Perms: drwxrwxr-x
/home            | Size: 68K        | Owner: root      | Group: root      | Perms: drwxr-xr-x
/usr/bin         | Size: 257M       | Owner: root      | Group: root      | Perms: drwxr-xr-x
/tmp             | Size: 28K        | Owner: root      | Group: root      | Perms: drwxrwxrwt
/usr/local       | Size: 100K       | Owner: root      | Group: root      | Perms: drwxr-xr-x
/opt             | Size: 4.0K       | Owner: root      | Group: root      | Perms: drwxr-xr-x

═══════════════════════════════════════════════════════════════════
                  SECTION 2: PYTHON DIRECTORIES AUDIT
═══════════════════════════════════════════════════════════════════

Python Installation Directories:
───────────────────────────────────────────────────────────────────
✓ /usr/lib/python3                | 119M | root:root | drwxr-xr-x
✗ /usr/lib/python3.11             | Not found
✓ /usr/lib/python3.12             | 54M | root:root | drwxr-xr-x
✗ /usr/local/lib/python3          | Not found
✓ /etc/python3                    | 8.0K | root:root | drwxr-xr-x
✗ /home/aryan/.local/lib/python   | Not found
✗ /home/aryan/.local/bin          | Not found

═══════════════════════════════════════════════════════════════════
                  SECTION 3: PYTHON EXECUTABLES
═══════════════════════════════════════════════════════════════════

Executable       | Location                                | Permissions
───────────────────────────────────────────────────────────────────
python           | Not installed
python3          | /usr/bin/python3                        | lrwxrwxrwx
python2          | Not installed
pip              | /usr/bin/pip                            | -rwxr-xr-x
pip3             | /usr/bin/pip3                           | -rwxr-xr-x

═══════════════════════════════════════════════════════════════════
                  SECTION 4: DISK SPACE SUMMARY
═══════════════════════════════════════════════════════════════════

Filesystem Disk Usage:

Filesystem      Size  Used Avail Use% Mounted on
/dev/sdd       1007G  2.7G  953G   1% /
tmpfs           780M   20K  780M   1% /run/user/1000

═══════════════════════════════════════════════════════════════════
                  SECTION 5: SECURITY ANALYSIS
═══════════════════════════════════════════════════════════════════

Checking for world-writable directories (potential security concern):

⚠  /tmp is potentially world-writable: drwxrwxrwt

Note: /tmp permissions: drwxrwxrwt
  (World-writable with sticky bit is expected and secure for /tmp)

═══════════════════════════════════════════════════════════════════
Audit completed at 14:45:50 on 28/03/2026
═══════════════════════════════════════════════════════════════════
```

Script 4: Log File Analyzer

This script reads a log file line by line, counts occurrences of a specified keyword (defaulting to 'error'), and provides a summary with the last matching lines. It includes Python-related log entry detection and multi-keyword analysis capabilities. The script demonstrates while-read loops with IFS handling, command-line argument processing ($1, $2), counter variables, grep for pattern matching, and tail for displaying last lines. The script accepts a log file path as an argument and an optional keyword parameter.

```bash
#!/bin/bash
# ============================================================
# Script 4: Log File Analyzer
# Author: Aryan Shandilya | Course: Open Source Software      You, 24 minutes ago
# Description: Reads a log file line by line, counts occurrences
#              of a keyword, handles missing/empty files, and
#              prints the last 5 matching lines.
# Usage: ./script4_log_analyzer.sh <logfile> [keyword]
# ============================================================

# --- Command-line arguments ---
# $1 = log file path (required), $2 = keyword (optional, defaults to "error")
LOGFILE=$1
KEYWORD=${2:-"error"}   # Use "error" as default if no keyword given

# --- Counter variable to track keyword matches ---
COUNT=0

echo "================================================================"
echo "              LOG FILE ANALYZER"
echo "================================================================"
echo "  Log file : ${LOGFILE:-[not specified]}"
echo "  Keyword  : $KEYWORD"
echo "----------------------------------------------------------------"

# --- Validate that a log file argument was provided ---
if [ -z "$LOGFILE" ]; then
    echo "[ERROR] No log file specified."
    echo "Usage: $0 <logfile> [keyword]"
    echo "Example: $0 /var/log/syslog error"
    exit 1
fi

# --- Do-while style retry: attempt to find/wait for a non-empty file ---
# Bash does not have native do-while; we simulate it with a loop + break
MAX_RETRIES=3  # Maximum number of retry attempts
```

```
RETRY=0        # Current retry count

while true; do
    # Check if file exists at all
    if [ ! -f "$LOGFILE" ]; then
        echo "[ERROR] File not found: $LOGFILE"
        RETRY=$((RETRY + 1))  # Increment retry counter

        # If we've exhausted retries, exit with an error
        if [ "$RETRY" -ge "$MAX_RETRIES" ]; then
            echo "[FAIL] File not found after $MAX_RETRIES attempts. Exiting."
            exit 1
        fi

        echo "[RETRY $RETRY/$MAX_RETRIES] Waiting 2 seconds before retry..."
        sleep 2   # Wait before retrying
        continue  # Go back to top of loop
    fi

    # Check if the file is empty
    if [ ! -s "$LOGFILE" ]; then
        echo "[WARN] File exists but is empty: $LOGFILE"
        RETRY=$((RETRY + 1))

        if [ "$RETRY" -ge "$MAX_RETRIES" ]; then
            echo "[FAIL] File remained empty after $MAX_RETRIES retries. Exiting."
            exit 1
        fi

        echo "[RETRY $RETRY/$MAX_RETRIES] File is empty, retrying in 2 seconds..."
        sleep 2
        continue
    fi

```

```
    # File exists and is non-empty — break out of the retry loop
    break
done

echo "[OK] File found and readable. Scanning..."
echo ""

# --- while-read loop: read the file line by line ---
# IFS= preserves leading/trailing whitespace in each line
# -r prevents backslash interpretation
while IFS= read -r LINE; do
    # if-then: check if this line contains the keyword (case-insensitive via -i)
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))  # Increment counter for each matching line
    fi
done < "$LOGFILE"   # Redirect file into the while loop (not a pipe, preserves variables)

# --- Summary output ---
echo "  Total lines scanned : $(wc -l < "$LOGFILE")"
echo "  Keyword '$KEYWORD'  : found $COUNT time(s)"
echo ""

# --- Show the last 5 matching lines for context ---
echo "----------------------------------------------------------------"
echo "  Last 5 lines containing '$KEYWORD':"
echo "----------------------------------------------------------------"

# grep -i = case-insensitive | tail -5 = last 5 results
MATCHES=$(grep -i "$KEYWORD" "$LOGFILE" | tail -5)

if [ -n "$MATCHES" ]; then
    # Loop through matched lines and print with indentation
    while IFS= read -r MATCH_LINE; do
        echo "  >> $MATCH_LINE"
    done <<< "$MATCHES"
else
    echo "  [NONE] No lines matched keyword '$KEYWORD'."
fi

echo "================================================================"
```

Given Screenshot:-

```
============================================================
            LOG FILE ANALYZER
============================================================
  Log file : [not specified]
  Keyword  : error
------------------------------------------------------------
[ERROR] No log file specified.
Usage: ./script4_log_analyzer.sh <logfile> [keyword]
Example: ./script4_log_analyzer.sh /var/log/syslog error

C:\Users\aryan\OneDrive\Desktop\CODING\CLONED REPO's\OSS-project-24MIM10151>wsl bash ./script4_log_analyzer.sh /var/log/bootstrap.log error
============================================================
            LOG FILE ANALYZER
============================================================
  Log file : /var/log/bootstrap.log
  Keyword  : error
------------------------------------------------------------
[OK] File found and readable. Scanning...

  Total lines scanned : 1494
  Keyword 'error'  : found 5 time(s)


------------------------------------------------------------
  Last 5 lines containing 'error':
------------------------------------------------------------
  >> 2025-08-05 16:55:04 URL:http://ftpmaster.internal/ubuntu/pool/main/libg/libgpg-error/libgpg-error0_1.47-3build2_amd64.deb [69990/69990] -> "/build/chro
ot//var/cache/apt/archives/partial/libgpg-error0_1.47-3build2_amd64.deb" [1]
  >> Selecting previously unselected package libgpg-error0:amd64.
  >> Preparing to unpack .../libgpg-error0_1.47-3build2_amd64.deb ...
  >> Unpacking libgpg-error0:amd64 (1.47-3build2) ...
  >> Setting up libgpg-error0:amd64 (1.47-3build2) ...
============================================================
```

Script 5: OPEN SOURCE MANIFESTO GENERATOR

This creative script generates a personalized open – source philosophy statement by interactively asking the user three questions and composing a manifesto saved to a timestamped text file. It demonstrates read for interactive user input, string concatenation for text composition, file redirection (> and >>) for output, date command for timestamps, and includes documentation of the alias concept. The generated manifesto includes the Four Freedoms of free software and a personal pledge section, demonstrating how shell scripting can be used for both technical and creative purposes.

```
#!/bin/bash
# ==============================================================
# Script 5: Open Source Manifesto Generator
# Author: Aryan Shandilya | Course: Open Source Software      You, 26 minutes ago • First Up
# Description: Interactively generates a personalised open-source
#              philosophy statement from user responses and saves
#              it to a .txt file.
# ==============================================================

# --- Alias demonstration (as a comment, since aliases do not export to scripts) ---
# In a normal shell session, a user might set:
#   alias manifesto='bash ~/scripts/script5_manifesto_generator.sh'
# This would allow running the script simply by typing: manifesto
# Aliases are shell conveniences — they do not persist in child scripts.

echo "================================================================"
echo "      OPEN SOURCE MANIFESTO GENERATOR"
echo "      Powered by your beliefs, not a corporation."
echo "================================================================"
echo ""
echo "Answer three questions honestly to generate your personal"
echo "open-source philosophy manifesto."
echo ""

# --- read: capture user input interactively ---
# -p displays a prompt inline before reading input
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

# --- Variables for metadata ---
DATE=$(date '+%d %B %Y')          # e.g. "20 March 2026"
AUTHOR=$(whoami)                   # Username of the person running the script
OUTPUT="manifesto_${AUTHOR}.txt"  # Output filename derived from username

```

```
echo ""
echo "Generating your manifesto..."
echo ""

# --- String concatenation to compose the manifesto paragraph ---
# Using heredoc (<<) for clean multi-line string writing
# > creates/overwrites the file; >> appends (used for subsequent lines)

# Write the manifesto to the output file
{
    echo "================================================================"
    echo "  MY OPEN SOURCE MANIFESTO"
    echo "  Author : $AUTHOR"
    echo "  Date   : $DATE"
    echo "================================================================"
    echo ""
    echo "I am a part of a world built on openness. Every day, I rely on"
    echo "$TOOL — a tool that someone chose to build in the open and share"
    echo "freely, not for profit, but because they believed that knowledge"
    echo "belongs to everyone. That act of generosity shapes everything I do."
    echo ""
    echo "To me, freedom means $FREEDOM. In the context of software, that"
    echo "word is not abstract — it is the right to read the code that runs"
    echo "my computer, to change it when it does not serve me, and to share"
    echo "those changes with others who might benefit. Without that freedom,"
    echo "technology becomes a cage instead of a tool."
    echo ""
    echo "I commit to contributing to this tradition. One day, I will build"
    echo "$BUILD and release it openly — not to gain credit, but because"
    echo "the open-source community gave me everything I know, and the only"
    echo "honest response to that gift is to pass it forward."
    echo ""
    echo "The software I write will be free as in freedom."
    echo ""
    echo "  — $AUTHOR, $DATE"
    echo "================================================================"
} > "$OUTPUT"   # > redirects the entire block output into the file

# --- Confirm and display the saved manifesto ---
echo "Manifesto saved to: $OUTPUT"
echo ""

# cat: display the file contents to the terminal
cat "$OUTPUT"
```

Given Screenshot:-

```
================================================================
      OPEN SOURCE MANIFESTO GENERATOR
      Powered by your beliefs, not a corporation.
================================================================

Answer three questions honestly to generate your personal
open-source philosophy manifesto.

1. Name one open-source tool you use every day: python
2. In one word, what does 'freedom' mean to you? independence
3. Name one thing you would build and share freely: ai

Generating your manifesto...

Manifesto saved to: manifesto_aryan.txt

================================================================
  MY OPEN SOURCE MANIFESTO
  Author : aryan
  Date   : 28 March 2026
================================================================

I am a part of a world built on openness. Every day, I rely on
python - a tool that someone chose to build in the open and share
freely, not for profit, but because they believed that knowledge
belongs to everyone. That act of generosity shapes everything I do.

To me, freedom means independence. In the context of software, that
word is not abstract - it is the right to read the code that runs
my computer, to change it when it does not serve me, and to share
those changes with others who might benefit. Without that freedom,
technology becomes a cage instead of a tool.

I commit to contributing to this tradition. One day, I will build
ai and release it openly - not to gain credit, but because
the open-source community gave me everything I know, and the only
honest response to that gift is to pass it forward.

The software I write will be free as in freedom.

  - aryan, 28 March 2026
```

Script	Command
System Identity	./script1_system_identity.sh
Package Inspector	./script2_foss_package_inspector.sh
Disk Auditor	./script3_disk_permission_auditor.sh
Log Analyzer	./script4_log_file_analyzer.sh /var/log/syslog [keyword]
Manifesto Generator	./script5_manifesto_generator.sh

Table 4: Script Execution Commands

Conclusion

This project has provided an opportunity to conduct a comprehensive audit of Python, examining its origins, licensing, ethical dimensions, technical implementation, and place within the broader open-source ecosystem. Through this process, I have gained not only technical knowledge about how Python works on Linux systems, but also a deeper appreciation for the philosophical principles that drive open-source development.

Python's journey from a Christmas holiday hobby project to one of the world's most important programming languages illustrates the transformative potential of open-source software. Its permissive licensing has enabled adoption across industry, academia, and individual developers, while its community-driven governance ensures that it continues to evolve in ways that serve its diverse user base. The shell scripts developed as part of this project demonstrate practical Linux skills while embodying the spirit of automation and transparency that defines the open-source philosophy.

As software continues to permeate every aspect of modern life, the principles exemplified by Python—transparency, collaboration, accessibility, and community—become increasingly important. Whether one is a professional developer, a student, or simply a citizen in a digitized society, understanding open-source software is essential for navigating and shaping the technological landscape. This project has been a step toward that understanding, and I am grateful for the opportunity to have engaged with these ideas in depth.